Project - High Level Design

On

Autonomous Retail Researcher Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Tanish Israni	EN22IT301112
2	Samarth Patidar	EN22IT301087
3	Yash Dashore	EN22CS3011100
4	Ramakant Singh	EN22IT301077
5	Abhishek Gupta	EN22EL301004
6	Sourabh Patel	EN22IT301106

Group Name: Group 10D12

Project Number: AAI-01

Industry Mentor Name: Ashwin sir

University Mentor Name: Ajeet Singh Rajput

Academic Year: 2026

Table of Contents

1. **Introduction :**
   1.1. **Scope of the document :**

This document provides a high-level design of the *Autonomous Retail Research Agent*, an Agentic AI system that automates retail market research using multiple intelligent agents. The system leverages Large Language Models (LLMs), external search tools, and internal knowledge bases to generate structured research reports, including insights, trends, and visualizations.

The scope includes system architecture, workflows, components, APIs, data handling, and non-functional requirements

   1.2. **Intended Audience :**

- Students and developers working on the project
- Faculty and evaluators
- Industry mentors
- AI/ML practitioners interested in Agentic AI systems

   1.3. **System overview :**

The system is a multi-agent AI application built using:

- CrewAI framework for agent orchestration
- Streamlit for frontend UI
- LLM (via API) for reasoning and content generation
- Serper API for real-time web search
- Local knowledge base for domain-specific insights

Key Features:

- Autonomous research using AI agents
- Integration of internal + external knowledge
- Automated report generation (PDF + UI display)
- Data visualization (charts/graphs)
- Persistent storage of reports

2. **System Design :**
   2.1. **Application Design :**

The system follows a **layered architecture**:

1. Presentation Layer

- Built using Streamlit
- Handles user input (research query)
- Displays generated reports and charts

2. Agent Layer

- Uses CrewAI agents:
- Research Agent
- Analysis Agent
- Report Generation Agent

3. Service Layer

- LLM API calls
- Search API (Serper)
- Data processing logic

4. Data Layer

- Knowledge Base (text files)
- Internal Repository (generated reports)

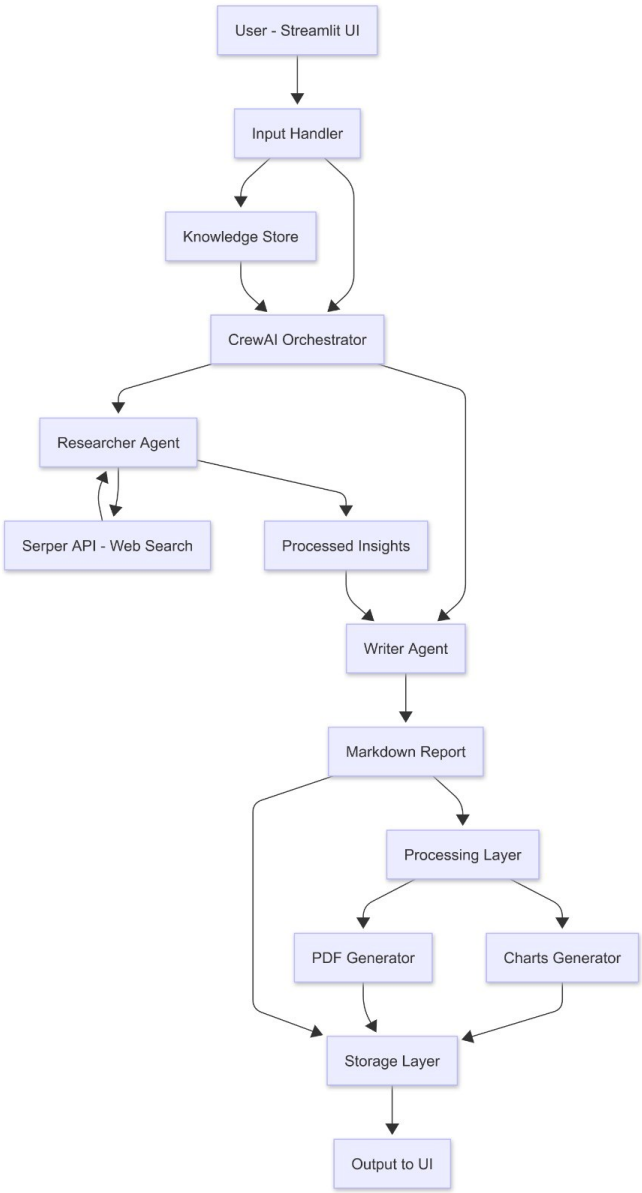

Fig 2.1 : Application Design

2.2. Process Flow :

1. User enters a retail research query
2. System initializes agents using CrewAI
3. Research Agent:
   - Fetches real-time data using search API
   - Extracts relevant information
4. Analysis Agent:
   - Processes collected data
   - Generates insights and trends
5. Report Agent:
   - Structures content into sections
   - Generates final report
6. Visualization:
   - Charts created using matplotlib
7. Output:
   - Displayed in UI
   - Saved as PDF in repository

2.3. Information Flow :

User → Streamlit UI → CrewAI Agents
→ (Search API + Knowledge Base)
→ Data Processing → Report Generation
→ Visualization → Output (UI + PDF Storage

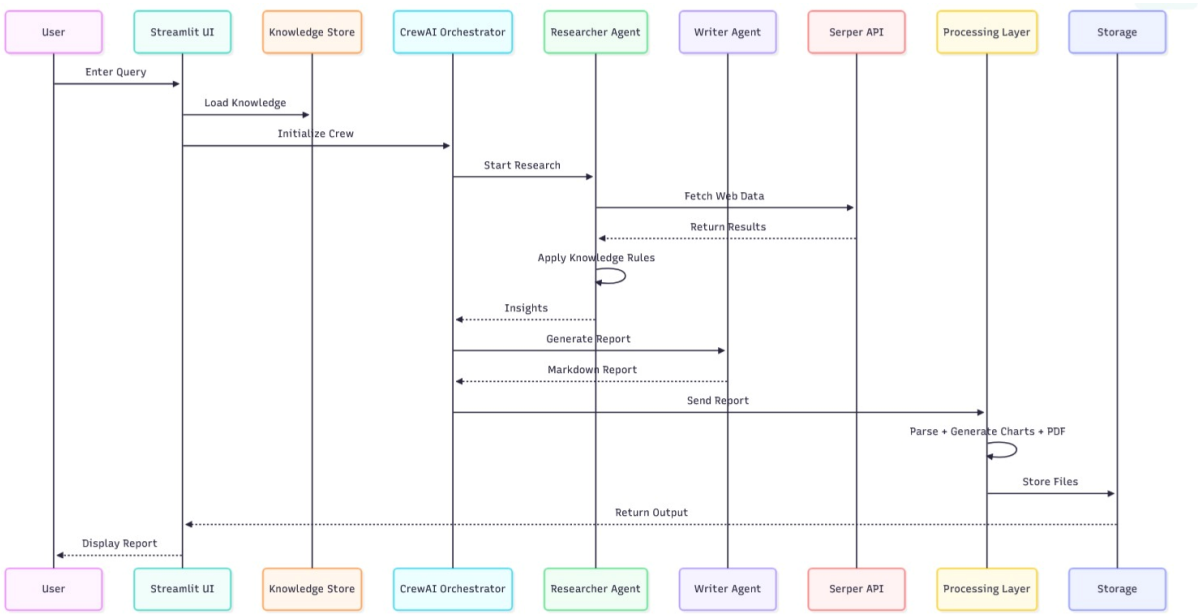

Fig 2.3 : Information Flow

2.4. Components Design :

1. Streamlit App (app.py)
   - Handles UI and workflow execution
   - Coordinates all components
2. Agents (CrewAI)
   - Defined using Agent, Task, and Crew
   - Responsible for autonomous decision-making
3. Knowledge Base
   - Stored in /knowledge_base/
   - Contains domain-specific retail data
4. External Tools
   - SerperDevTool for web search
   - LLM for reasoning
5. Report Generator
   - Uses ReportLab
   - Generates structured PDF reports

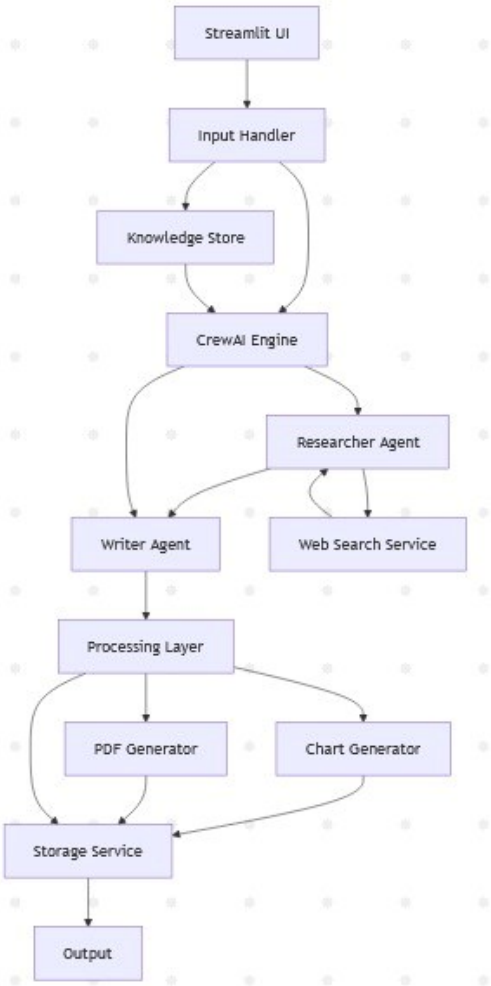

Fig 2.4 : Component Design

2.5. Key Design Considerations :

- **Scalability:** Modular agent-based design
- **Extensibility:** Easy addition of new agents/tools
- **Reusability:** Knowledge base reuse
- **Performance:** Efficient API calls
- **User Experience:** Clean UI with structured output
- **Automation:** Minimal manual intervention

2.6. API Catalogue :

1. LLM API :

- Used for text generation and reasoning
- Input: Prompt
- Output: Generated text

2. Serper Search API :

- Used for real-time web search
- Input: Query
- Output: Search results

2.7. Class Diagram :

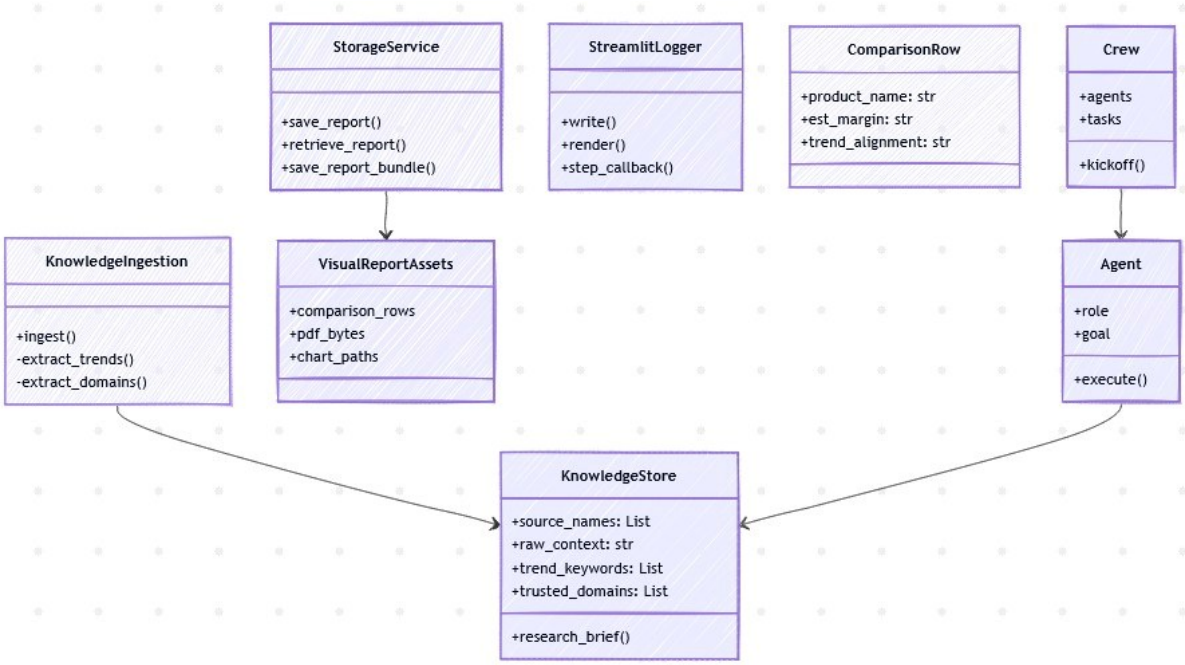

Fig 2.7 : Class Diagram

3. Data Design.

3.1. Data Model :

Unstructured Data:

- Text files (knowledge base)
- Web search results

Structured Data:

- Generated reports
- Analytical summaries

3.2. Data Access Mechanism :

- DirectoryLoader for reading files
- PDF/Text loaders for processing
- In-memory processing for agents

3.3. Data Retention Policies :

- Reports stored in internal repository
- Old reports retained for reference
- No sensitive personal data stored

3.4. Data Migration :

- Not required currently
- Future scope: Database integration

4. Interfaces :

The Autonomous Retail Research Agent interacts with multiple internal and external interfaces to ensure smooth functioning of the system.

1. User Interface

The system provides a web-based interface built using Streamlit. Users can enter research queries related to retail markets, such as trends, competitor analysis, or product insights. The interface displays structured reports, summaries, and visualizations like charts and graphs. It is designed to be simple and user-friendly so that even non-technical users can easily interact with the system.

2. External Interfaces

The system integrates with external APIs to enhance its capabilities:

- **LLM API:** Used for generating natural language responses, insights, and reports.
- **Search API (Serper):** Fetches real-time data from the web to ensure up-to-date research results.

These APIs are accessed securely using API keys and environment variables.

3. Internal Interfaces

Internal communication happens between different modules such as agents, data loaders, and report generators. The CrewAI framework enables structured communication between agents where each agent performs a specific task and passes results to the next stage.

5. **State and Session Management :**
State and session management are handled using Streamlit's built-in session state mechanism. This ensures that user interactions and generated outputs persist during a session.

When a user submits a query, the system stores:
- The input query
- Intermediate agent outputs
- Final generated report

This helps in maintaining continuity, especially when multiple steps are involved in report generation. For example, once the research process starts, intermediate data from the research agent and analysis agent is preserved and passed forward.

Session state also helps in:
- Avoiding repeated computations
- Allowing users to view previously generated results
- Managing UI updates dynamically

However, the session is temporary and resets when the user refreshes or closes the application. Persistent storage of final reports is handled separately in the internal repository.

6. **Caching :**
Caching is implemented to improve system performance and reduce redundant API calls. Since generating reports involves multiple API requests (LLM + search), caching helps in storing previously computed results.
When a user submits a query that has already been processed earlier:
- The system retrieves stored results instead of recomputing
- This significantly reduces response time

Caching is particularly useful for:
- Frequently searched retail topics
- Repeated queries during testing or demonstrations

Streamlit caching mechanisms (like st.cache_data) can be used to store:
- API responses
- Processed data
- Generated insights

This not only improves speed but also reduces API usage costs.

7. Non-Functional Requirements

7.1 Security Aspects :

Security is an important aspect of the system, especially when dealing with external APIs. The following measures are implemented:

- API keys are stored securely in .env files and not hardcoded
- Sensitive information is not exposed in the frontend
- Secure communication is ensured when calling external APIs
- The system does not store any personal or confidential user data

Additionally, access to the application can be restricted if deployed in a production environment.

7.2 Performance Aspects :

The system is designed to provide efficient and responsive performance despite involving multiple AI agents and API calls.

Key performance considerations include:

- **Parallel Processing:** Agents can work in sequence efficiently, reducing overall delay
- **Optimized API Calls:** Only necessary data is fetched and processed
- **Caching:** Reduces repeated computations and speeds up responses
- **Efficient Data Handling:** Lightweight data structures are used for faster processing

Even though LLM-based systems may introduce some latency, the design ensures that response time remains acceptable for users. Future improvements can include asynchronous processing and better scaling mechanisms.

8. References :

1. CrewAI Documentation
2. Streamlit Documentation
3. LangChain Community Loaders
4. Reportlab Documentation